

Cloud Computing Lab 5 – Assignment 1

Deploying and Connecting Amazon RDS (PostgreSQL) to the Lab 4 EC2 Microblog Application

Application	Microblog (Flask)
Database	Amazon RDS – PostgreSQL
RDS Instance Identifier	microblog-lab5-rds
EC2 Application URL	http://3.111.37.11
AWS Region	ap-south-1 (Asia Pacific – Mumbai)
GitHub Repository	github.com/appisal/aws-cloud-lab5-microblog

1. Architecture

The Microblog Flask application, deployed on an AWS EC2 instance in Lab 4, was connected to a dedicated Amazon RDS PostgreSQL instance. The application communicates with the database over PostgreSQL port 5432 within the same VPC. Database access is restricted at the network layer so that only the EC2 instance's security group may reach the database – the internet has no direct path to the RDS endpoint.

Internet → EC2 (Microblog Flask App) → PostgreSQL:5432 → Amazon RDS PostgreSQL

2. RDS Database Configuration

Setting	Value
Database Engine	PostgreSQL
DB Instance Identifier	microblog-lab5-rds
Instance Class	db.t4g.micro
Availability	Single-AZ
Port	5432
Master Username	microblogadmin
Region / AZ	ap-south-1b

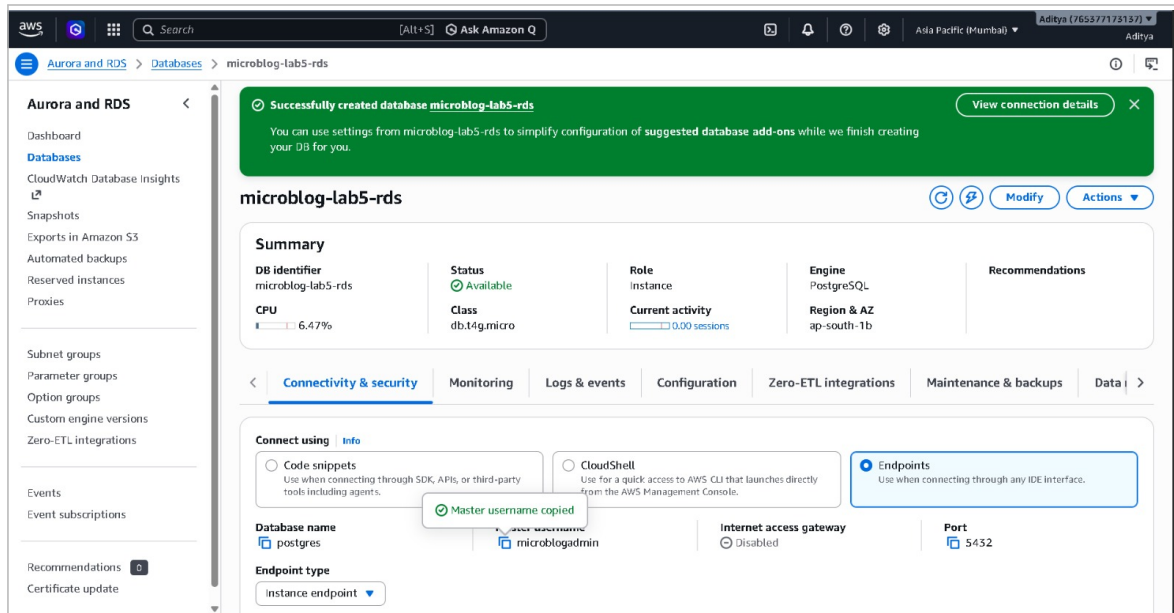


Figure 1: Amazon RDS console – microblog-lab5-rds instance, PostgreSQL engine, status Available, class db.t4g.micro

3. Security Configuration

The RDS security group (**microblog-rds-sg**, sg-0e10c6ab8134a3c37) was configured with an inbound rule of type **“EC2 Security Group – Inbound”**, sourced from the EC2 instance's security group (sg-02cf2dd53d435d8df) on port 5432. No inbound rule permits 0.0.0.0/0 (unrestricted internet) access, satisfying the requirement that database access be limited to the application's EC2 Security Group only.

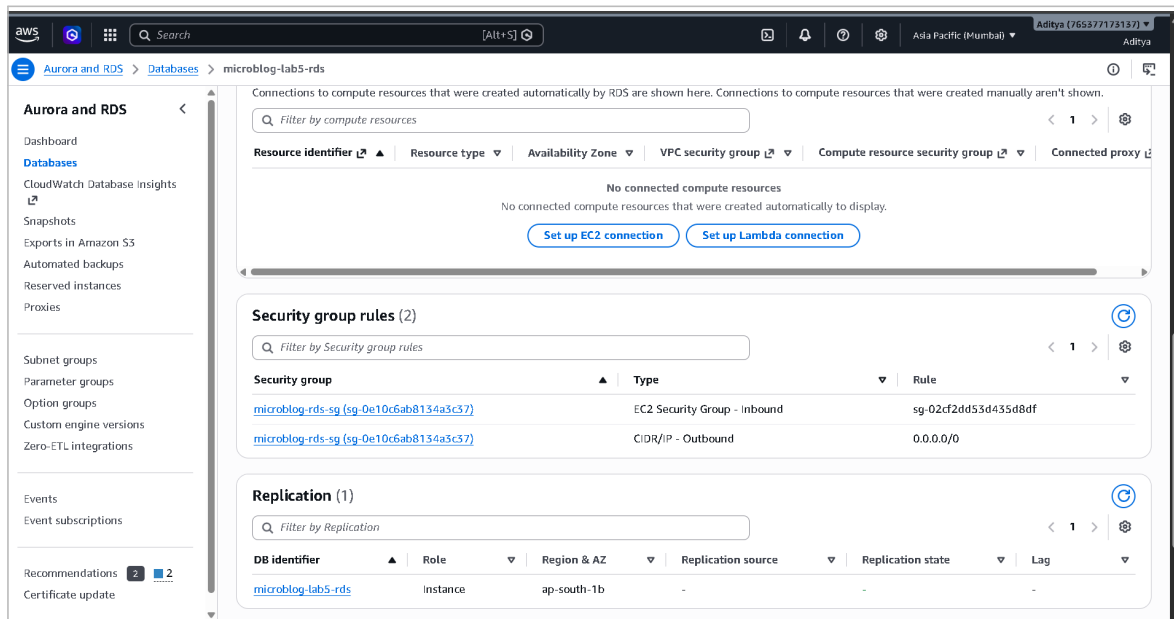


Figure 2: RDS security group rules – inbound rule type “EC2 Security Group - Inbound” sourced from sg-02cf2dd53d435d8df (no 0.0.0.0/0 inbound rule present)

4. Database Schema

The application's PostgreSQL schema was verified directly via `psql` after connecting to the **microblog** database on the RDS endpoint. The schema contains seven tables, exceeding the minimum requirement of two related tables:

Table	Purpose
user	Registered application users

post	User posts (foreign key & rarr; user)
followers	User-to-user follow relationships
message	Private messages between users
notification	User notifications
task	Background task tracking
alembic_version	Schema migration version (Flask-Migrate)

The **post** table maintains a foreign-key relationship to the **user** table (`user_id` → `user.id`), and the **followers** table defines a many-to-many self-relationship between users, together demonstrating a fully relational schema.

```

postgres=> \dt
Did not find any tables.
postgres=> \l
postgres=> \l
postgres=> \c microblog
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2), server 18.3)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "microblog" as user "microblogadmin".
microblog=> \dt
          List of tables
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | alembic_version | table | microblogadmin
public | followers       | table | microblogadmin
public | message         | table | microblogadmin
public | notification    | table | microblogadmin
public | post            | table | microblogadmin
public | task            | table | microblogadmin
public | user            | table | microblogadmin
(7 rows)

```

Figure 3: psql session connected to the microblog database on RDS – \dt output listing all 7 schema tables

5. CRUD Demonstration (from the running EC2 application)

Operation	Application Action	Result Observed
Create	Submitted a new post as user aditya3	“Your post is now live!” – post displayed in feed
Read	Feed loaded after posting	New post visible under aditya3's profile
Update	Edited the post text	“Your post has been updated.” – new text shown
Delete	Deleted the post	“Your post has been deleted.” – post removed from feed

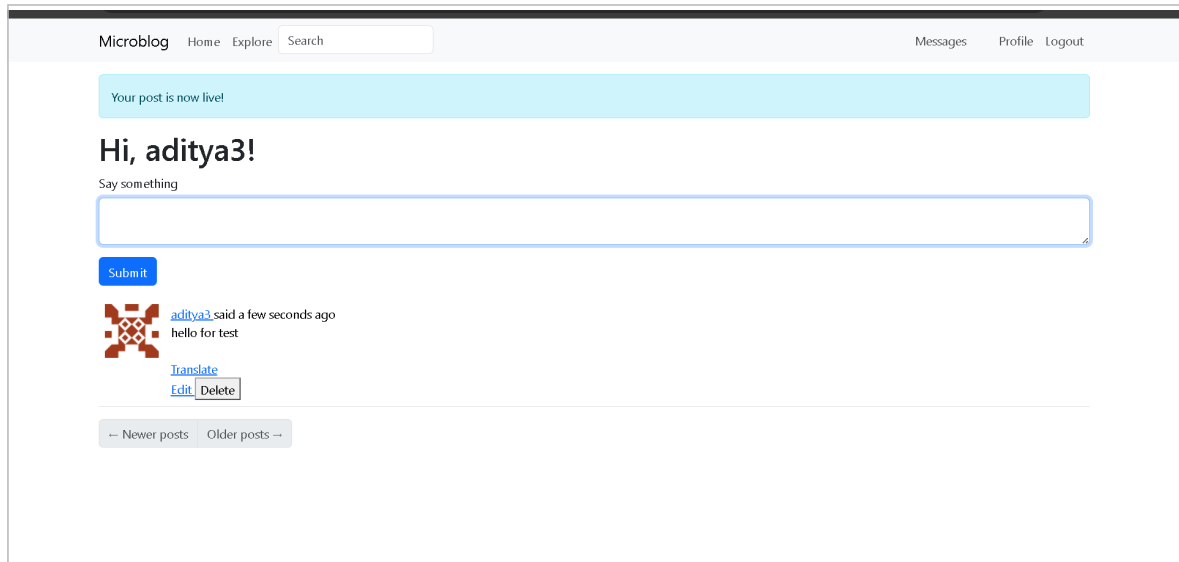


Figure 4: Create – user aditya3 submits a new post; “Your post is now live!” confirmation shown

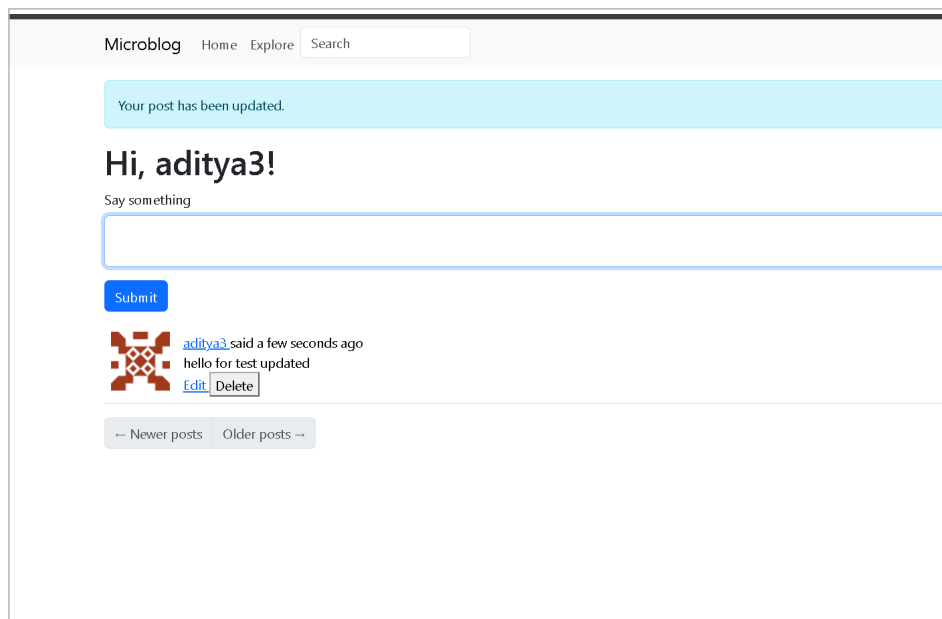


Figure 5: Update – post text edited; “Your post has been updated.” confirmation shown

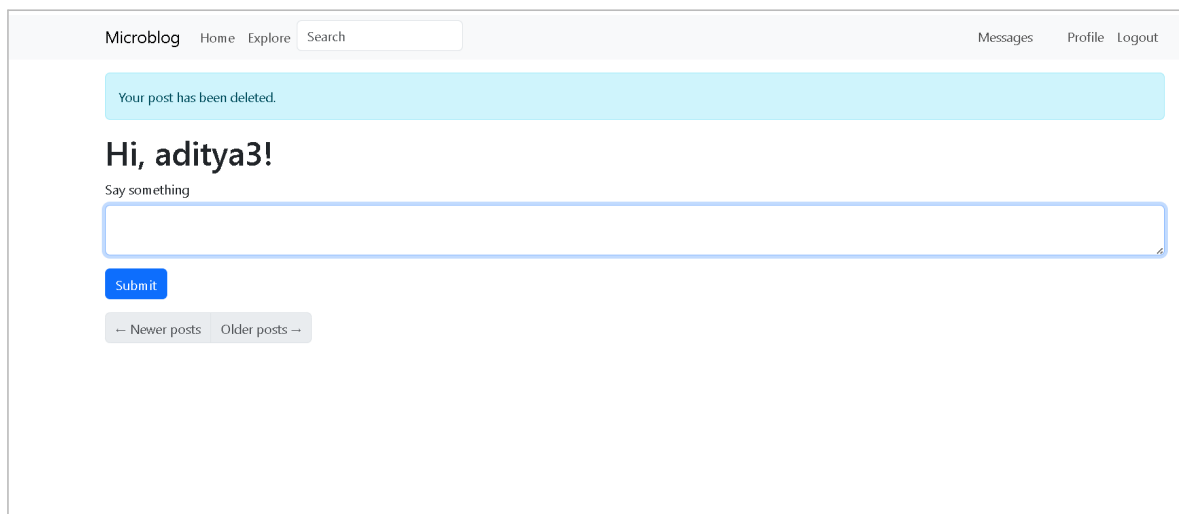


Figure 6: Delete – post removed; “Your post has been deleted.” confirmation shown

6. Evidence Submitted

- RDS instance dashboard showing engine, status Available, and instance class

- Security group rules screenshot confirming inbound restricted to EC2 Security Group
- psql session output (\dt) listing all 7 schema tables after connecting to the microblog database
- Application screenshots: post created, post updated, post deleted (user aditya3)

7. Deliverables

- GitHub Repository: github.com/appisal/aws-cloud-lab5-microblog
- EC2 Application URL: <http://3.111.37.11>
- Database: Amazon RDS PostgreSQL (microblog-lab5-rds)
- CRUD: Create, Read, Update, Delete – all demonstrated from the live application
- Security: RDS inbound access restricted to the EC2 Security Group only (no 0.0.0.0/0)